

What are Micro-Partitions?

In Snowflake, all table data is **automatically divided into smaller storage units called Micro-Partitions**.

Micro-partitions are the **physical storage layer of Snowflake tables**.

Each micro-partition stores **50 MB – 500 MB of uncompressed data** and the data is stored in **columnar format**.

This architecture helps Snowflake achieve **high performance, compression, and efficient query processing**.

Important point:

Snowflake automatically manages micro-partitions, so users **do not need to create or manage partitions manually**.

Key Characteristics of Micro-Partitions

1 Automatic Partitioning

When data is loaded into a Snowflake table, Snowflake automatically divides it into micro-partitions.

No manual partitioning is required.

This is different from traditional databases where users must define partitions manually.

2 Columnar Storage

Micro-partitions store data in **columnar format** instead of row format.

Benefits of columnar storage:

- Faster analytical queries
- Higher compression
- Efficient scanning of required columns only

This design is ideal for **data warehouse workloads**.

3 Metadata Storage

Snowflake stores **metadata for each micro-partition**, such as:

- Minimum value of column
- Maximum value of column
- Number of distinct values

This metadata helps Snowflake **skip unnecessary partitions during queries**.

What is Partition Pruning?

Partition Pruning is a performance optimization technique.

When a query runs, Snowflake checks the **metadata of micro-partitions**.

Instead of scanning the entire table, Snowflake scans **only the required micro-partitions**.

Example:

Table contains **10 micro-partitions**.

Query:

```
SELECT *  
FROM orders  
WHERE order_date = '2025-01-01'
```

Snowflake checks metadata and scans **only partitions containing that date**.

This reduces:

- Query execution time
- Data scanning cost

Pruning is extremely important for **Data Warehouse query optimization**.

Micro-Partitions are Immutable

Micro-partitions are **immutable**, meaning they **cannot be modified once created**.

When a record is updated:

1. Snowflake creates a **new micro-partition**
2. Updated row is written into the new partition
3. Old partition is marked for deletion

This design supports Snowflake features like:

- Time Travel
 - Fail-safe
 - Zero-copy cloning
-

Example of Update Operation

Suppose we have a table:

Loan_Dimension

LoanID Customer Amount

1	John	5000
---	------	------

LoanID Customer Amount

2	Mike	8000
3	David	10000

If we update:

```
UPDATE Loan_Dimension  
SET Amount = 9000  
WHERE LoanID = 2;
```

Process:

1. Snowflake creates a **new micro-partition**
2. Updated row is written to the new partition
3. Old partition becomes inactive

This ensures **data consistency and historical tracking**.

Constraints in Snowflake

Snowflake allows defining constraints like:

- Primary Key
- Foreign Key
- Unique Key

However:

These constraints are **not enforced**.

They are mainly used for **logical documentation and query optimization**.

The only constraint that Snowflake enforces is:

NOT NULL constraint

Example:

Loan_Dimension table

Primary Key defined on:

LoanID → 1,2,3

Even if duplicates exist, Snowflake **will not block the insert**.

Snowflake vs Traditional Databases

Traditional databases (OLTP systems):

- Designed for record-by-record transactions
- Use indexes and row-based storage

Snowflake (Data Warehouse):

- Designed for **big data analytics**
 - Optimized for **bulk data loads and large queries**
 - Uses **micro-partitions instead of indexes**
-

When Micro-Partitions are Created

Micro-partitions are created during:

- Data loading
- INSERT operations
- COPY INTO commands
- Table updates

Snowflake continuously reorganizes partitions for **optimal performance**.

Advantages of Micro-Partitions

- Automatic partition management
 - Faster query performance
 - Reduced data scanning
 - Better compression
 - Efficient analytics processing
 - Supports large-scale data warehouses
-

Key Interview Points

Micro-partition size: **50 MB – 500 MB**

Storage type: **Columnar**

Micro-partitions are **immutable**

Supports **partition pruning**

Snowflake automatically manages partitioning

Primary and foreign keys are **not enforced**

Only **NOT NULL constraint is enforced**

Snowflake is optimized for **OLAP workloads, not OLTP transactions**

Quick Summary

Micro-partitions are the **core storage architecture of Snowflake**.

They divide large tables into **small optimized columnar storage units**, enabling:

- Faster queries
- Efficient data scanning
- Automatic performance optimization

This design is one of the **main reasons Snowflake performs extremely well for analytical workloads**.

ROHITH KUMAR